

C6P/IslandAccord/Convro — Threat Model (v1)

Security Analysis for Auditors

Convro Protocol Team

January 2026

1 Executive Summary

C6P (Convro 6 Protocol) is an end-to-end encrypted direct messaging protocol featuring IslandAccord v1 handshake (3DH+OTP) and symmetric double-ratchet with per-message forward secrecy.

Core principles: - **Server is not trusted for secrecy:** routes messages, never derives keys - **Deterministic crypto by design:** derived nonces, no RNG in AEAD - **Fail-closed as invariant:** any validation failure aborts immediately

This document analyzes: handshake replay, OTP exhaustion, state machine races, message transport attacks.

Explicitly out of scope: endpoint compromise, malware, social engineering, quantum adversaries, DoS.

2 System Overview & Trust Boundaries

2.1 2.1 Actors

- **Initiator (A):** Creates session, owns `IK_sig`, `IK_dh`, generates ephemeral `A_EK`
- **Responder (B):** Accepts session, owns identity keys + rotating prekeys (SPK, OTP)
- **Server (S):** Routes messages, enforces state machine, **never derives secrets**
- **Network Attacker:** Observes/modifies transport, sees metadata (session ID, timestamps, sizes)

2.2 2.2 Trust Boundaries

- **Client<->Server:** Server authoritative for routing/state, NOT trusted for crypto validation
- **Client<->Network:** E2E encrypted payloads, metadata visible (session ID, size, timing)
- **Server DB:** Atomic state transitions, immutable offer/accept blobs

2.3 2.3 Assets

- **Session secrecy:** Root key, chain keys -> compromise reveals future messages
 - **Forward secrecy:** Per-message keys deleted after use
 - **OTP scarcity:** Single-use guarantee enforced by atomic DB transitions
 - **State machine integrity:** Invalid transitions rejected (fail-closed)
 - **Replay resistance:** Consumed counter sets prevent message duplication
-

3 Attacker Model

3.1 3.1 Attacker Capabilities

- **Passive observer:** Record encrypted traffic (harvest-now-decrypt-later)
- **Active MITM:** Intercept, modify, replay, drop, reorder messages
- **Malicious server:** Serve fake prekeys, deny service, analyze metadata
- **Malicious peer:** Leak own session keys (forward secrecy limits damage)
- **Compromised device (limited):** Access keys on device; past messages protected by forward secrecy

3.2 3.2 What Attacker CANNOT Do (Assumptions)

- Break cryptographic primitives (ChaCha20-Poly1305, Ed25519, X25519)
 - Extract keys from uncompromised device secure storage (Keychain/KeyStore)
 - Forge Ed25519 signatures without private keys
-

4 Security Goals

1. **Confidentiality:** Only intended recipients decrypt message content
 2. **Authentication:** Peers verify identities via Ed25519 signatures + KC tags
 3. **Replay protection:** Handshake and messages reject duplicates (consumed sets)
 4. **State machine integrity:** Server enforces transitions with fail-closed validation
 5. **OTP single-use guarantee:** Atomic consumption tracking prevents reuse
 6. **Deterministic reproducibility:** All operations testable with fixed inputs
-

5 Threat Enumeration Methodology

Threats grouped by **protocol phase**: - **Handshake**: Offer/accept construction, KC validation, OTP lifecycle - **Server State Machine**: Transition enforcement, concurrency races - **Message Transport**: Replay, reordering, counter manipulation

Analysis follows **STRIDE-inspired** approach adapted for E2EE: - **Spoofing**: Identity substitution, SPK replay - **Tampering**: Offer modification, KC tag manipulation - **Replay**: Message/handshake duplication - **Information Disclosure**: Metadata leakage (session linkability) - **Denial of Service**: OTP exhaustion, counter overflow (bounded) - **Elevation**: Invalid state transitions

6 Threat Scenarios (15)

6.1 Handshake / Session (1-5)

6.1.1 Threat 1: Replayed Handshake Offer

- **Attacker:** Network MITM
- **Goal:** Reuse captured offer to initiate duplicate session
- **Attack Path:** (1) Capture valid offer, (2) Replay to server after original session created
- **Affected Phase:** Handshake (OPEN -> PENDING_ACCEPT)
- **Mitigation:** SessionId uniqueness (§4.1 island-accord-crypto.md), server rejects duplicate sessionId
- **Residual Risk:** None
- **Status:** Prevented

6.1.2 Threat 2: Modified Offer Blob

- **Attacker:** Malicious server
- **Goal:** Tamper with offer to alter DH contributions or suite
- **Attack Path:** (1) Intercept offer, (2) Modify `A_EK` or `suite_id`, (3) Forward to responder
- **Affected Phase:** Handshake (responder validates offer)
- **Mitigation:** Offer signature binds initiator `IK_sig` to entire offer blob (§3.2 island-accord-crypto.md), transcript hash includes all fields
- **Residual Risk:** None
- **Status:** Prevented

6.1.3 Threat 3: SessionId Collision Attempt

- **Attacker:** Malicious initiator
- **Goal:** Force collision with existing sessionId to hijack session
- **Attack Path:** (1) Generate predictable sessionId, (2) Submit offer with collision
- **Affected Phase:** Handshake (server state machine)
- **Mitigation:** Server rejects duplicate sessionId (§2.1 island-accord-state-machine.md), 64-bit random sessionId has negligible collision probability
- **Residual Risk:** None (2^{64} space)
- **Status:** Prevented

6.1.4 Threat 4: Accept Replay with Modified KC2

- **Attacker:** Malicious server
- **Goal:** Replay accept with altered KC2 to establish session without peer agreement
- **Attack Path:** (1) Capture valid accept, (2) Modify KC2, (3) Deliver to initiator
- **Affected Phase:** Handshake (KC validation)
- **Mitigation:** KC2 derived from transcript hash (§5 island-accord-crypto.md), any modification fails HMAC verification
- **Residual Risk:** None
- **Status:** Prevented

6.1.5 Threat 5: Cross-Device Accept Attempt

- **Attacker:** Malicious responder
- **Goal:** Accept offer from device B1 using device B2's keys
- **Attack Path:** (1) Offer sent to B1, (2) B2 attempts accept with different `responder_device_id`
- **Affected Phase:** Handshake (device binding)
- **Mitigation:** Offer specifies `responder_device_id` (§3.1 island-accord-crypto.md), server validates device match
- **Residual Risk:** None
- **Status:** Prevented

6.2 OTP / Scarcity (6-9)

6.2.1 Threat 6: OTP Reuse Attempt

- **Attacker:** Malicious server
- **Goal:** Serve same OTP to multiple initiators to compromise uniqueness
- **Attack Path:** (1) Reserve OTP for A1, (2) Serve same OTP in bundle to A2
- **Affected Phase:** Handshake (bundle fetch)
- **Mitigation:** Atomic state transitions (§3.2 island-accord-state-machine.md), OTP marked **RESERVED** -> **CONSUMED** atomically
- **Residual Risk:** None
- **Status:** Prevented

6.2.2 Threat 7: OTP Race Between Initiators

- **Attacker:** Network timing
- **Goal:** Two initiators fetch same OTP simultaneously
- **Attack Path:** (1) A1 and A2 call `/fetch-bundle` concurrently, (2) Both receive same OTP
- **Affected Phase:** Handshake (concurrent fetch)
- **Mitigation:** DB-level row locking on `OTP SELECT FOR UPDATE` (§3.3 island-accord-state-machine.md), only one transaction succeeds
- **Residual Risk:** None
- **Status:** Prevented

6.2.3 Threat 8: OTP Replay After Expiration

- **Attacker:** Malicious initiator
- **Goal:** Reuse expired OTP to establish session
- **Attack Path:** (1) Reserve OTP, (2) Wait for TTL expiry, (3) Submit offer with expired OTP
- **Affected Phase:** Handshake (OTP validation)
- **Mitigation:** Server checks OTP status and expiry (§3.4 island-accord-state-machine.md), rejects `CONSUMED` or expired OTPs
- **Residual Risk:** None
- **Status:** Prevented

6.2.4 Threat 9: OTP Prefetch Hoarding

- **Attacker:** Malicious initiator
- **Goal:** Reserve all OTPs to deny service to legitimate users
- **Attack Path:** (1) Call `/fetch-bundle` repeatedly, (2) Exhaust OTP pool
- **Affected Phase:** Handshake (resource exhaustion)
- **Mitigation:** Rate limiting on `/fetch-bundle` (§6.1 island-accord-observability.md), reservation TTL (5 min) releases unused OTPs
- **Residual Risk:** Limited (DoS mitigated by rate limits)
- **Status:** Detected + Mitigated

6.3 Server State Machine (10-12)

6.3.1 Threat 10: Invalid State Transition Injection

- **Attacker:** Malicious client
- **Goal:** Force session from `OPEN` directly to `ACTIVE` bypassing `accept`
- **Attack Path:** (1) Submit offer, (2) Submit message without `accept`
- **Affected Phase:** Server state machine
- **Mitigation:** State machine validates transitions (§2.2 island-accord-state-machine.md), `OPEN -> ACTIVE` only via `PENDING_ACCEPT -> ACTIVE`
- **Residual Risk:** None
- **Status:** Prevented

6.3.2 Threat 11: Double Accept Concurrency

- **Attacker:** Malicious responder
- **Goal:** Submit two accepts for same offer to create duplicate sessions
- **Attack Path:** (1) Responder submits `accept_1`, (2) Before commit, submits `accept_2`
- **Affected Phase:** Handshake (accept processing)
- **Mitigation:** DB transaction isolation (§3.5 island-accord-state-machine.md), only first accept commits
- **Residual Risk:** None
- **Status:** Prevented

6.3.3 Threat 12: Offer Delivery After Terminal State

- **Attacker:** Network delay
- **Goal:** Deliver offer to session already in COMPLETED/FAILED
- **Attack Path:** (1) Session reaches terminal state, (2) Delayed offer arrives
- **Affected Phase:** Server state machine
- **Mitigation:** Terminal states reject all transitions (§2.3 island-accord-state-machine.md), session immutable after COMPLETED
- **Residual Risk:** None
- **Status:** Prevented

6.4 Network / Transport (13-15)

6.4.1 Threat 13: Message Replay on WebSocket Reconnect

- **Attacker:** Network MITM
- **Goal:** Replay captured message after WS reconnect
- **Attack Path:** (1) Capture encrypted envelope, (2) Replay after connection drop
- **Affected Phase:** Message transport
- **Mitigation:** Consumed counter set (§4 dm-ratchet-state-machine.md), duplicate counter rejected
- **Residual Risk:** None
- **Status:** Prevented

6.4.2 Threat 14: Counter Desync Attack

- **Attacker:** Malicious server
- **Goal:** Deliver messages out-of-order to desync counter expectations
- **Attack Path:** (1) Buffer messages, (2) Deliver in reversed order
- **Affected Phase:** Message transport
- **Mitigation:** Skip-window (2048) accepts out-of-order within bounds (§3.1 c6p-replay-and-skip-window.md), consumed set prevents replay
- **Residual Risk:** Limited (bounded reordering allowed by design)
- **Status:** Detected + Bounded

6.4.3 Threat 15: Malicious Server Message Reordering

- **Attacker:** Malicious server
- **Goal:** Reorder messages to confuse client state
- **Attack Path:** (1) Hold msg_10, (2) Deliver msg_11 first, (3) Deliver msg_10
- **Affected Phase:** Message transport
- **Mitigation:** Out-of-order tolerance within skip-window (§3.2 c6p-replay-and-skip-window.md), counters ensure causality detection
- **Residual Risk:** Limited (intentional design for unreliable transport)
- **Status:** Tolerated (by design)

7 Non-Goals & Explicit Limitations

C6P does not address:

1. **Compromised Endpoint:** Malware, keyloggers, malicious OS—rely on platform security (Keychain, biometrics)
2. **Physical Device Theft:** If device unlocked, attacker has full access—out of scope for protocol
3. **Side-Channel Attacks:** Power analysis, cache timing—mitigate with constant-time crypto libraries
4. **Denial of Service:** Network flooding, server overload—handle with operational defenses (rate limits, firewalls)

5. **Metadata Leakage:** Session IDs, message sizes, timestamps visible to network—accept as protocol limitation (future: padding, cover traffic)
6. **Social Engineering:** User tricked into accepting fake fingerprint—rely on user education
7. **Legal Coercion:** Government forces key disclosure—out of scope; consider plausible deniability in v2
8. **Quantum Adversaries:** X25519 vulnerable to Shor’s algorithm—acknowledged; v2 will support hybrid post-quantum prekeys

These are not failures—they are explicit boundaries. Auditors should assess C6P within its defined threat model, not against unbounded adversaries.

8 Determinism & Fail-Closed Philosophy

8.1 Why Deterministic Nonces Are Safe

- **Traditional AEAD:** Random nonce + key -> ciphertext (nonce collision catastrophic)
- **C6P Design:** Nonce derived from `mk_material` (§7 c6p-key-schedule.md) via `HKDF-Expand(mk_material, "C6P_NONCE_V1")`
- **Safety:** Each `mk_material` unique per message, nonce collision impossible

Benefits: Cross-platform reproducibility, no RNG dependency, testable with fixed vectors.

8.2 Fail-Closed Invariants

1. **Unknown inputs abort:** Unrecognized suite ID, invalid version -> immediate rejection
2. **Validation failures abort:** Bad signature, KC mismatch, replay detected -> session terminated
3. **State never advances on error:** DB transaction rollback, no partial updates

Philosophy: Better to fail loudly than silently accept insecure state.

9 Conclusion & Security Posture

C6P v1 is **secure within its defined threat model**:

- ☑ Handshake authenticated (Ed25519 + KC tags)
- ☑ Messages encrypted end-to-end (per-message AEAD)
- ☑ Replay prevented (consumed sets)
- ☑ Forward secrecy achieved (ratcheting)
- ☑ OTP scarcity enforced (atomic state transitions)
- ☑ Fail-closed validation (no silent failures)

Residual risks (acknowledged): - Endpoint compromise (out of scope) - Metadata visible to network (v1 limitation) - Quantum vulnerability (addressed in v2)

Recommendation: C6P suitable for production deployment after external cryptographic audit. Protocol design is sound; implementation quality determines real-world security.

10 Appendix A: Threat Summary Table

ID	Threat	Attacker	Status	Residual Risk
1	Replayed offer	Network	Prevented	None

ID	Threat	Attacker	Status	Residual Risk
2	Modified offer blob	Server	Prevented	None
3	SessionId collision	Malicious initiator	Prevented	None
4	Accept replay (KC2)	Server	Prevented	None
5	Cross-device accept	Malicious responder	Prevented	None
6	OTP reuse	Server	Prevented	None
7	OTP race	Concurrent clients	Prevented	None
8	OTP replay (expired)	Malicious initiator	Prevented	None
9	OTP prefetch hoarding	Malicious initiator	Mitigated	Limited (rate limits)
10	Invalid state transition	Malicious client	Prevented	None
11	Double accept	Malicious responder	Prevented	None
12	Offer after terminal	Network delay	Prevented	None
13	Message replay (WS)	Network	Prevented	None
14	Counter desync	Server	Bounded	Limited (skip-window)
15	Message reordering	Server	Tolerated	By design

11 Appendix B: Glossary

- **IK:** Identity Key (long-term Ed25519/X25519 keys)
- **SPK:** Signed Prekey (30-day rotation, Ed25519 signature)
- **OTP:** One-Time Prekey (single-use X25519 key)
- **KC:** Key Confirmation (HMAC tags proving mutual key agreement)
- **DH:** Diffie-Hellman key exchange
- **AEAD:** Authenticated Encryption with Associated Data (ChaCha20-Poly1305)
- **Skip-window:** Out-of-order message acceptance (2048-message bound)
- **Consumed set:** Bitmap tracking received message counters (replay prevention)

Document Version: 1.0 **Date:** January 2026 **Status:** Production / Normative **Contact:** security@convro.protocol (audit inquiries)